

STRUTTURE DATI ELEMENTARI - ESERCIZI

PIETRO DI LENA

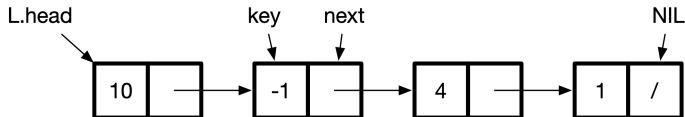
DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



ESERCIZIO 1

- Scrivere un algoritmo che, dati in input un intero positivo k ed una **lista concatenata semplice**, restituisca il k -esimo elemento a partire dalla fine
- Esempio, se $k = 1$ l'algoritmo deve restituire l'ultimo elemento; se $k = 2$ il penultimo, e così via



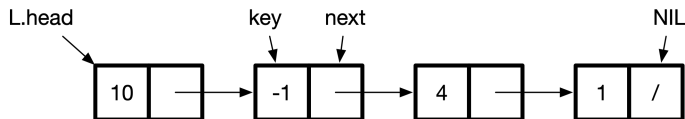
ESERCIZIO 1 - SOLUZIONE

```
1: function LASTK(LIST  $L$ , INT  $k$ )  $\rightarrow$  NODE
2:    $tmp = L.head$                                  $\triangleright$  Temporary pointer
3:    $n = 0$                                            $\triangleright$  Number of nodes
4:   while  $tmp \neq \text{NIL}$  do                       $\triangleright$  Count the number of nodes
5:      $n = n + 1$ 
6:      $tmp = tmp.next$ 
7:   if  $n < k$  then
8:     return NIL
9:   else
10:     $tmp = L.head$                                  $\triangleright$  Reset tmp
11:     $n = n - k + 1$                                  $\triangleright$  Index of last-k node
12:     $i = 1$ 
13:    while  $i < n$  do                               $\triangleright$  Search the last-k node
14:       $tmp = tmp.next$ 
15:       $i = i + 1$ 
16:    return  $tmp$ 
```

- Costo computazionale: $\Theta(n)$ (visitiamo sempre tutta L : linee 4-6)
- n = numero di nodi nella lista

ESERCIZIO 2

- Scrivere un algoritmo che, dati in input un intero positivo k ed una **lista concatenata semplice** restituisca il k -esimo elemento a partire dalla fine **senza visitare per due volte la lista** (per memorizzarne la lunghezza)
- Esempio, se $k = 1$ l'algoritmo deve restituire l'ultimo elemento; se $k = 2$ il penultimo, e così via



ESERCIZIO 2 - SOLUZIONE

```
1: function LASTK(LIST  $L$ , INT  $k$ )  $\rightarrow$  NODE
2:    $tmp1 = L.head$   $\triangleright$  Temporary pointer
3:    $tmp2 = L.head$   $\triangleright$  Temporary pointer
4:    $i = 0$   $\triangleright$  Counter
5:   while  $tmp1 \neq \text{NIL}$  and  $i < k$  do  $\triangleright$  Move  $tmp2$  on the  $k$ -th node
6:      $i = i + 1$ 
7:      $tmp1 = tmp1.next$ 
8:   if  $i < k$  then  $\triangleright$  There are less than  $k$  nodes
9:     return NIL
10:  else
11:     $\triangleright$  The distance between  $tmp1$  and  $tmp2$  is exactly  $k$ 
12:    while  $tmp1 \neq \text{NIL}$  do
13:       $tmp1 = tmp1.next$ 
14:       $tmp2 = tmp2.next$ 
15:    return  $tmp2$ 
```

- Costo computazionale: $\Theta(n)$ ($tmp1$ visita sempre tutti i nodi di L)
- n = numero di nodi nella lista

ESERCIZIO 3

- Scrivere un algoritmo **ricorsivo** che, data una **lista concatenata semplice**, la modifichi eliminando ogni elemento pari
- Esempio, se $L = [4, 6, 7, 3, 2, 5]$ al termine dell'esecuzione $L = [7, 3, 5]$

ESERCIZIO 3 - SOLUZIONE

```
1: function DELETEEVEN(NODE  $L$ )  $\rightarrow$  NODE
2:   if  $L == \text{NIL}$  then
3:     return NIL
4:   else if  $L.\text{key} \bmod 2 == 0$  then                                 $\triangleright$  Even node, remove
5:     return DELETEEVEN( $L.\text{next}$ )
6:   else                                                             $\triangleright$  Odd node, no remove
7:      $L.\text{next} = \text{DELETEEVEN}(L.\text{next})$ 
8:     return  $L$ 
```

- Costo computazionale: $\Theta(n)$ (visita sempre tutti i nodi della lista)

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

- n = numero di nodi nella lista

ESERCIZIO 4

- Scrivere un algoritmo **ricorsivo** che, data una **lista concatenata semplice**, la modifichi eliminando ogni elemento pari e replicando ogni elemento dispari tante volte quanti sono gli elementi pari che lo precedono
- Esempio, se $L = [4, 6, 7, 3, 2, 5]$ al termine dell'esecuzione abbiamo che $L = [7, 7, 7, 3, 3, 3, 5, 5, 5, 5]$

ESERCIZIO 4 - SOLUZIONE 1

```
1: function DELETEANDDUPLICATE(Node L, INT nPrec) → NODE
2:   if L == NIL then
3:     return NIL
4:   else if L.key mod 2 == 0 then                                ▷ Even node, remove
5:     return DELETEANDDUPLICATE(L.next, nPrec + 1)
6:   else                                                         ▷ Odd node, duplicate
7:     L.next = DELETEANDDUPLICATE(L.next, nPrec)
8:     while nPrec > 0 do                                         ▷ Duplication with nPrec head insert
9:       tmp = NODE(L.key)
10:      tmp.next = L
11:      L = tmp
12:      nPrec = nPrec - 1
13:   return L
```

- Soluzione mista ricorsiva/iterativa
- Prima esecuzione: DELETEANDDUPLICATE(L, 0)

ESERCIZIO 4 - SOLUZIONE 2

```
1: function DELETEANDDUPLICATE(NODE  $L$ , INT  $nPrec$ , INT  $nDup$ )  $\rightarrow$  NODE
2:   if  $L == \text{NIL}$  then
3:     return NIL
4:   else if  $L.key \bmod 2 == 0$  then ▷ Even node, remove
5:     return DELETEANDDUPLICATE( $L.next, nPrec + 1, nPrec + 1$ )
6:   else if  $nPrec > 0$  then ▷ Odd node, duplicate
7:      $tmp = \text{NODE}(L.key)$ 
8:      $tmp.next = \text{DELETEANDDUPLICATE}(L, nPrec, nPrec - 1)$ 
9:     return  $tmp$ 
10:  else ▷ Odd node, no duplicate
11:     $L.next = \text{DELETEANDDUPLICATE}(L.next, nPrec, nPrec)$ 
12:    return  $L$ 
```

- Soluzione interamente ricorsiva
- Prima esecuzione: $\text{DELETEANDDUPLICATE}(L, 0, 0)$

ESERCIZIO 4 - ANALISI DEL COSTO COMPUTAZIONALE

- Vale sia per la versione mista che per la versione solo ricorsiva
- Assumiamo n = numero di nodi nella lista
- Costo ottimo: $\Theta(n)$ (ci sono solo nodi pari o solo nodi dispari)
- Caso pessimo (irrealistico): ogni nodo viene duplicato tante volte quanti sono i nodi che lo precedono

$$T(n) \leq \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} = O(n^2)$$

Quindi il costo pessimo $T(n)$ è limitato superiormente da $O(n^2)$

- Caso reale: un nodo pari ed un nodo dispari, alternati

$$T'(n) = \sum_{i=1}^{n/2} i = \frac{n/2(n/2 + 1)}{2} = \frac{n^2}{8} + \frac{n}{4} = \Theta(n^2)$$

- Poiché $T'(n) = O(T(n))$ concludiamo che il costo pessimo è $\Theta(n^2)$

ESERCIZIO 5

- Scrivere un algoritmo che, presi in input una **lista concatenata semplice** con chiavi intere ed un intero x , calcoli il numero di nodi con chiave x
- Scrivere un algoritmo ricorsivo ed uno iterativo

ESERCIZIO 5 - SOLUZIONE

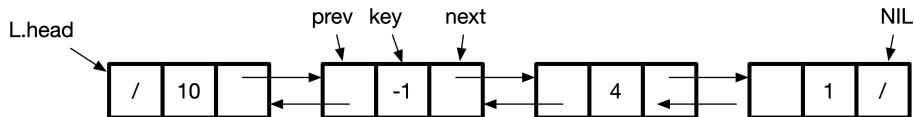
```
1: function COUNT(LIST  $L$ , INT  $x$ )  $\rightarrow$  INT  $\triangleright$  Soluzione iterativa  
2:    $tmp = L.head, n = 0$   
3:   while  $tmp \neq \text{NIL}$  do  
4:     if  $tmp.key == x$  then  
5:        $n = n + 1$   
6:      $tmp = tmp.next$   
7:   return  $n$ 
```

```
1: function COUNT(NODE  $L$ , INT  $x$ )  $\rightarrow$  INT  $\triangleright$  Soluzione ricorsiva  
2:   if  $L == \text{NIL}$  then  
3:     return 0  
4:   else if  $L.key == x$  then  
5:     return  $1 + \text{COUNT}(L.next)$   
6:   else  
7:     return COUNT( $L.next$ )
```

- Costo computazionale: $\Theta(n)$, n = numero di nodi nella lista
- Entrambe le implementazioni devono visitare tutti i nodi nella lista

ESERCIZIO 6

- Scrivere l'algoritmo INSERTIONSORT per **liste doppiamente concatenate**



ESERCIZIO 6 - SOLUZIONE

```
1: function INSERTIONSORT(DLLIST L)
2:   tmp = L.head
3:   while tmp ≠ NIL do
4:     p = tmp
5:     while p.prev ≠ NIL and p.key < p.prev.key do
6:       SWAP(p,p.prev)
7:       p = p.prev
8:     tmp = tmp.next
9:
10: function SWAP(DLLIST p, DLLIST q)
11:   if p ≠ NIL and q ≠ NIL then
12:     tmp = p.key, p.key = q.key, q.key = tmp
```

- Il ciclo while a riga 3 viene eseguito n volte
- Caso ottimo (ciclo while a riga 4 mai eseguito): $\Theta(n)$
- Caso pessimo (ciclo while a riga 4 eseguito interamente):

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2)$$

ESERCIZIO 7

Consideriamo la seguente sequenza di caratteri

E A S * Y * Q U E * * * S T * * * I O * N * * *

- 1 Partendo da una pila vuota S , un carattere c indica l'operazione $push(S, c)$ mentre un asterisco $*$ indica l'operazione $pop(S)$. Mostrare la sequenza di caratteri ritornata dalle operazioni pop .
- 2 Partendo da una coda vuota Q , un carattere c indica l'operazione $enqueue(Q, c)$ mentre un asterisco $*$ indica l'operazione $dequeue(Q)$. Mostrare la sequenza di caratteri ritornata dalle operazioni $dequeue$.

ESERCIZIO 7 - SOLUZIONE

- 1 Sequenza di caratteri ritornata dalle operazioni *pop*

S Y E U Q T S A O N I

- 2 Sequenza di caratteri ritornata dalle operazioni *dequeue*

E A S Y Q U E S T I O N

ESERCIZIO 8

- Consideriamo una pila S che supporta le seguenti operazioni in $O(1)$
 - $\text{TOP}(S)$ ritorna il valore in cima alla pila S (senza rimuoverlo)
 - $\text{ISEMPTY}(S)$ true se S è vuota, falso altrimenti
 - $\text{PUSH}(S, x)$ inserisce x in cima alla pila S
 - $\text{POP}(S)$ rimuove e ritorna il valore in cima alla pila S
 - Termina in errore se S è vuota (ex., lancia un'eccezione)
- Vogliamo mantenere traccia del valore minimo in una pila S_1
- Usiamo una seconda pila S_2 che ha sempre in cima il valore attualmente minimo in S_1
- Ideare le seguenti due operazioni, con costo $O(1)$
 - 1 $\text{PUSH2}(S_1, S_2, x)$ inserisce x in S_1 e aggiorna (se necessario) la cima di S_2 in modo che contenga il valore minimo in S_1
 - 2 $\text{POP2}(S_1, S_2)$ rimuove e ritorna il valore in cima ad S_1 e aggiorna (se necessario) S_2

ESERCIZIO 8 - SOLUZIONE

```
1: function PUSH2(STACK  $S_1$ , STACK  $S_2$ , INT  $x$ )  
2:   PUSH( $S_1$ ,  $x$ )  
3:   if ISEMPY( $S_2$ ) or TOP( $S_2$ )  $\geq x$  then  
4:     PUSH( $S_2$ ,  $x$ )
```

```
1: function POP2(STACK  $S_1$ , STACK  $S_2$ )  $\rightarrow$  INT  
2:   if not ISEMPY( $S_1$ ) and TOP( $S_1$ ) == TOP( $S_2$ ) then  
3:     POP( $S_2$ )  
4:   return POP( $S_1$ )
```

- PUSH2: se il nuovo valore x è più piccolo o uguale al valore in cima ad S_2 allora inserisce x in cima ad S_2
- POP2: tre casi possibili:
 - 1 il valore top in S_1 è uguale a quello in $S_2 \Rightarrow$ pop su S_2
 - 2 il valore top in S_1 è più grande di quello in $S_2 \Rightarrow$ nessuna azione
 - 3 il valore top in S_1 è più piccolo di quello in $S_2 \Rightarrow$ impossibile (il valore in cima ad S_2 è il minimo in S_1)
- Entrambe le funzioni usano solo operazioni $O(1) \Rightarrow$ costo $O(1)$

ESERCIZIO 9

- Scrivere lo pseudocodice di un algoritmo `MERGESTACKS( $S_1, S_2, S_3$ )` che, presi in input due pile **ordinate** S_1 and S_2 (elemento minimo in cima), costruisce S_3 , inizialmente vuota, con tutti gli elementi in S_1 e S_2 in ordine (dal più grande al più piccolo)
- Usare solo le operazioni della pila `ISEMPTY`, `TOP`, `PUSH`, `POP`
- Il costo pessimo di `MERGESTACKS` deve essere $O(n_1 + n_2)$ dove
 - n_1 = numero di elementi in S_1
 - n_2 = numero di elementi in S_2

ESERCIZIO 9 - SOLUZIONE

```
1: function MERGESTACKS(STACK  $S_1$ , STACK  $S_2$ , STACK  $S_3$ )
2:   if not ISEMPY( $S_1$ ) or not ISEMPY( $S_2$ ) then
3:     if ISEMPY( $S_1$ ) or TOP( $S_1$ )  $\geq$  TOP( $S_2$ ) then
4:        $x = \text{POP}(S_2)$ 
5:     else if ISEMPY( $S_2$ ) or TOP( $S_2$ )  $>$  TOP( $S_1$ ) then
6:        $x = \text{POP}(S_1)$ 
7:     MERGESTACKS( $S_1, S_2, S_3$ )
8:     PUSH( $S_3, x$ )
```

- Costo computazionale $\Theta(n_1 + n_2)$
 - La ricorsione termina quando sia S_1 che S_2 sono vuote
 - In ogni chiamata ricorsiva un solo valore viene rimosso o da S_1 (line 4) o da S_2 (line 6)
- Notiamo che la chiamata ricorsiva a riga 7, eseguita prima di PUSH a riga 8, assicura che gli elementi siano inseriti in S_3 partendo dal più grande al più piccolo
 - Invertendo le righe 7 e 8 otteniamo S_3 ordinato in modo inverso